SmartFire-NOC: Intelligent Fire Department Inspection and No-Objection Certificate Management System

line 1: 1st Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 2nd Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 3rd Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 4th Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 5th Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 6th Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

Abstract

Obtaining a fire No-Objection Certificate (NOC) involves identifying applicable requirements, submitting documents, scrutiny, inspection, rectification, and a final decision. Applicants may be unsure which requirements apply, submitted documents may be incomplete or inconsistent, and inspectors must organize limited time across many cases. This paper proposes SmartFire-NOC, an intelligent web-based decision-support platform for the Fire Department inspection and NOC process, developed as an academic prototype. The system combines a configurable rule-based checklist engine, an OCR- and NLP-based document verification pipeline, a limited computer-vision analysis of fire-safety drawings, a controlled provisional-to-final NOC lifecycle, a transparent weighted-priority inspection scheduler, and a tamper-evident audit trail based on SHA-256 hash chaining. It uses React, Node.js, Express, and MongoDB Atlas, with JWT authentication and role-based access control. AI outputs are suggestions for human review, compliance decisions rely on deterministic rules, and approval remains with authorized personnel. The system will be evaluated on synthetic and authorized data for checklist correctness, OCR and classification accuracy, symbol-detection precision and recall, scheduling performance against a baseline, workflow validity, and audit-tampering detection. SmartFire-NOC is an academic prototype and not an official government service.

Keywords— Fire NOC, Fire Safety Inspection, Workflow Automation, Optical Character Recognition, Document Verification, Computer Vision, Inspection Scheduling, Tamper-Evident Audit Log, Role-Based Access Control, Decision Support.

I. INTRODUCTION

Fire safety approval is a statutory requirement for many buildings. In Maharashtra, the Maharashtra Fire Prevention and Life Safety Measures Act, 2006 requires the owner or occupier of buildings listed in its schedule to obtain Provisional Fire Safety Approval, Final Fire Safety Approval, or renewal from the Chief Fire Officer of the concerned local or planning authority, or from the Director where no Chief Fire Officer exists [1]. The Act describes provisional approval as a recommendation given at the time of building plan approval and before construction, and final approval as a certificate issued after the authority ascertains that the measures conform to the provisional approval, before a completion or occupancy certificate is granted [1]. In practice these approvals are commonly handled as provisional and final Fire No-Objection Certificates (NOCs).

Several Indian states have moved parts of this process online. The DIGIT Fire NOC application offers online submission and fee payment, a role-based workflow that includes document scrutiny, field inspection, and approval, and certificates carrying QR codes [2]. The Telangana fire department portal allows provisional NOC, occupancy NOC, and renewal services to be requested online, lets applicants view inspection reports, and sends SMS alerts at different stages [3]. These systems show that digitizing submission, payment, and tracking is feasible and useful.

Digitizing forms, however, does not by itself reduce the analytical work around an application. Someone must still determine which requirements apply to a particular building, check whether uploaded documents are present, legible, and consistent with the application, examine drawings, decide which cases to inspect first, and keep a history that can be trusted. The public descriptions of the portals above emphasize submission, payment, workflow, and tracking [2], [3]. This paper does not claim that these systems lack further functions internally; it observes only that automated document-consistency checks, drawing analysis, transparent inspection prioritization, and tamper-evident auditing are not described there.

Research in artificial intelligence has addressed neighbouring problems. Machine learning has been used to prioritize fire inspections using historical property and incident data [4], [5]. Knowledge-driven pipelines have combined layout extraction, object detection, and regulatory clauses for fire-safety compliance checking [6]. Dedicated datasets exist for symbol spotting in CAD drawings [7], and multimodal Document AI models handle classification and information extraction from forms [8]. Each of these addresses one component. Among the sources reviewed here, none connects them within a single application-to-final-certificate workflow with human oversight.

Four practical difficulties motivate the design. First, an applicant often does not know in advance which requirements apply to a particular building, so applications are submitted incomplete and are returned. Second, documents such as ownership records, approved plans, and equipment certificates are checked largely by reading them, which is slow and prone to oversight when volumes are high. Third, inspectors have limited time and must decide in what order to visit cases, usually without a shared and visible rule. Fourth, once many officials and applicants touch the same record, it becomes hard to show later who did what and when. Each of the six modules answers one of these difficulties.

To address this gap, this paper proposes SmartFire-NOC, an intelligent web-based platform for Fire Department inspection and NOC management. The platform links six modules: a dynamic checklist engine, AI-assisted document verification, preliminary drawing analysis, provisional-to-final NOC tracking, smart inspection scheduling, and a digital audit trail. A guiding principle is that every advanced technology must solve a specific problem in the NOC process. Compliance decisions are made by deterministic, versioned rules and not by a language model; AI findings are suggestions for human review; and approval and issuance remain with authorized personnel. SmartFire-NOC is an academic decision-support prototype and has no official affiliation with the Government of Maharashtra or any fire department.

The main contributions of this work are:

- a configurable, versioned rule engine that generates a personalized, explainable NOC checklist;
- a document pipeline that turns OCR output into reviewable flags for missing, unreadable, and inconsistent documents;
- a controlled lifecycle that enforces permitted transitions and role permissions from application to final decision;
- a transparent, configurable inspection scheduler that respects statutory deadlines; and
- a SHA-256 hash-chained audit trail that makes changes to historical records detectable.

The remainder of the paper reviews related work, states the research gap, describes the methodology and architecture, presents the planned evaluation, discusses limitations and future scope, and concludes.

II. RELATED WORK / LITERATURE REVIEW

Related research falls into five groups: data-driven prioritization of fire inspections, automated fire-safety compliance checking, symbol detection in drawings, Document AI, and existing e-governance NOC systems. Work on tamper-evident logging is reviewed because the audit module depends on it. The studies below are summarized from their published abstracts and descriptions; their results are reported as stated by the authors.

A. Firebird: Predicting Fire Risk and Prioritizing Fire Inspections in Atlanta

Madaio et al. [4] developed Firebird with the Atlanta Fire Rescue Department, whose inspection practice had relied on tradition and intuition rather than a data-driven process. The framework combines machine learning, geocoding, and information visualization to identify and prioritize commercial properties for inspection. It builds fire-risk models, including Support Vector Machines and Random Forests, computes risk scores for over 5,000 buildings with true positive rates of up to 71% in predicting fires, and presents results on an interactive map. The study shows the value of data-driven prioritization. It selects properties by predicted fire risk, whereas SmartFire-NOC orders an existing queue of applications using administrative factors and does not predict fire risk.

Two methodological points from this study shape the present work. The first is that integration effort was a major part of the contribution: the framework joins several datasets, and the quality of the joined records limits the quality of the predictions. SmartFire-NOC faces a similar issue at a smaller scale, because the checklist engine, the document pipeline, and the scheduler all depend on clean and consistent application data. The second is the use of an interactive map as a decision aid and not as an automatic decision maker. The inspectors remain the users who decide. This matches the human-in-the-loop principle adopted here for all AI output.

B. Data-Driven Structural Fire Risk Prediction for City Properties

Dey and Fern [5] investigate how fire departments with limited inspection resources can decide which properties to inspect. Using data from a South Dakota fire department covering more than 72,000 properties, they train XGBoost models to predict a fire-risk score from historical incident records and evaluate how well the scores rank properties. The method is chosen to cope with missing data and a highly skewed class distribution, and the authors analyze feature importance. This work supports the use of ranking-based prioritization, but it needs historical incident data that a new prototype does not have. For this reason SmartFire-NOC starts from a transparent weighted score and treats risk-based prioritization as later work.

The study also demonstrates an evaluation habit worth copying. Because only a small share of properties experience fires, simple accuracy would be misleading, so the authors evaluate ranking quality and examine which features drive predictions. In SmartFire-NOC the same caution applies to document-mismatch detection and symbol detection, where the interesting cases (a real mismatch, a missing exit symbol) are rare compared with ordinary cases. For this reason the evaluation plan in Section VI reports precision, recall, and per-class results and not accuracy alone.

C. Knowledge-Driven Fire Safety Compliance Checking

Chen et al. [6] present a knowledge-based approach to fire-safety compliance monitoring in operational buildings. It integrates building layouts, in-situ images, and regulatory clauses in an architecture that combines multimodal perception, semantic integration, and rule-based inference. The reported experiments achieve pixel accuracy of 0.90 for layout extraction and mAP of 0.91 for safety-asset detection, and a domain-adapted visual question answering model improves compliance descriptions over a generic vision-language baseline. The study is relevant because it separates perception from rule-based reasoning, as SmartFire-NOC does. Its scope, which includes knowledge graphs and vision-language models for operational buildings, is broader than the limited symbol detection planned here, and it does not address the NOC application workflow.

An important lesson from this study is the separation of three tasks: seeing (perception of layouts and assets), linking (semantic integration with regulatory clauses), and deciding (rule-based inference). Where the decision step is made by explicit rules, each outcome can be traced to a clause, which supports review and correction. SmartFire-NOC applies the same idea in two places. The checklist engine decides applicability by explicit rules with source references, and the drawing module only reports what it detects, leaving the compliance judgment to the inspector. The study also shows that detection quality depends on the type of asset and image, which is why the present work restricts itself to a short list of symbols and reports results per class.

D. FloorPlanCAD: Panoptic Symbol Spotting in CAD Drawings

Fan et al. [7] introduce FloorPlanCAD, a large-scale CAD drawing dataset for panoptic symbol spotting. Its relevance is that symbol detection in architectural drawings is a recognized task with a dedicated benchmark. Its limits for this project are practical. SmartFire-NOC must accept PDFs and images that may be scanned rather than vector CAD files, and the symbol conventions in fire-safety drawings submitted to a given authority may differ from those in the dataset. The dataset is therefore a reference and a possible source for pre-training, while evaluation will use a separately annotated sample set.

Symbol spotting differs from ordinary object detection because drawing symbols are small, thin, often overlap with lines and text, and may be drawn differently by different designers. Scanned drawings add noise, skew, and compression artifacts. These properties explain why a model that works on clean vector drawings may degrade on scanned submissions, and why SmartFire-NOC flags low-confidence detections for manual review and does not attempt automatic rejection.

E. LayoutLMv3: Document AI

Huang et al. [8] propose LayoutLMv3, a multimodal Transformer pre-trained with unified text and image masking and a word-patch alignment objective. It reports state-of-the-art results on text-centric tasks such as form understanding, receipt understanding, and document visual question answering, and on image-centric tasks such as document image classification and layout analysis. It is a candidate model family for classifying and extracting fields from uploaded documents. These results come from public benchmarks and do not show how such a model behaves on ownership records, approved plans, or fire-equipment certificates, so SmartFire-NOC will measure its own accuracy and compare against simpler baselines.

Document AI also depends on the quality of the text recognition that precedes it. Layout-aware models need words and their positions from an OCR step, so OCR errors propagate into classification and extraction. For this reason the pipeline in Section IV records OCR confidence, measures character error rate separately, and routes unreadable or low-confidence documents to the applicant for resubmission instead of trusting the downstream result. It also keeps a simple baseline classifier, because a smaller model that is easier to explain and run may be adequate for a limited set of document categories.

F. Existing E-Governance Fire NOC Systems

The DIGIT Fire NOC application [2] provides online application and fee payment, role-based workflow, comments and documents captured at each workflow role, status dashboards, an application history, and extraction of system logs for analysis. The Telangana portal [3] was built to let applicants request provisional and occupancy NOCs and renewals online instead of visiting offices. These systems establish the administrative workflow that SmartFire-NOC builds on. SmartFire-NOC adds decision-support layers on top of that workflow and does not replace it, and it makes no claim to improve on these deployed systems without evaluation.

Two features of these systems are directly relevant. Both treat the application as a case that moves through roles, which supports modelling the process as a state machine. Both also expose history to the user, which supports the audit and status-tracking modules. Other states, such as Haryana and Madhya Pradesh, publish step-by-step Fire NOC procedures with document checklists and approval stages, which shows that the process is broadly similar across states but that details differ. This is the reason SmartFire-NOC keeps requirements and lifecycle steps as configuration and does not hard-code them.

G. Tamper-Evident Audit Logging

Schneier and Kelsey [9] describe a computationally inexpensive method for protecting log files on an untrusted machine, so that entries made before a compromise cannot be read, and cannot be modified or destroyed without detection. Their scheme is stronger than the basic hash chain planned here, because it also addresses confidentiality of earlier entries after compromise. SmartFire-NOC adopts only the tamper-evidence idea and states its limits plainly.

Hash chaining is the simplest member of this family of techniques. Each record includes a digest of its predecessor, so any change to an earlier record alters every later digest and can be found by recomputation. Its strength is low cost and easy verification; its weakness is that it protects nothing if an attacker can rewrite the whole chain. The design in Section IV therefore treats chaining as one layer, to be combined with restricted storage permissions and separately stored checkpoints.

H. Summary of the Literature

Table I summarizes the reviewed work. The studies show that data-driven prioritization, perception combined with rule-based reasoning, symbol spotting, Document AI, and tamper-evident logging are each feasible, and that online NOC portals exist. They also differ in task, data, and evaluation, so their results cannot be combined into an expected performance for a new system.

TABLE I. SUMMARY OF REVIEWED WORK

Ref.	Main approach	Relevance / limit for this work
[4]	ML fire-risk scores (SVM, Random Forest) and map for inspection prioritization	Prioritizes properties by predicted risk; not an application workflow
[5]	XGBoost fire-risk ranking on 72,000+ properties	Needs historical incident data
[6]	Perception, semantic integration, rule-based inference for compliance	Broader scope; no NOC lifecycle
[7]	Dataset for panoptic symbol spotting in CAD drawings	Different drawing formats and symbol sets
[8]	Multimodal Document AI model	Benchmark results; domain data differs
[2], [3]	Online Fire NOC portals with role-based workflow	Public descriptions emphasize submission and tracking
[9]	Cryptographic secure audit logs	Stronger than planned hash chain

III. RESEARCH GAP AND PROBLEM STATEMENT

A. Research Gap

The reviewed work leaves four gaps. First, published descriptions of online Fire NOC portals concentrate on submission, payment, workflow, and tracking [2], [3]. They do not describe automated checks of whether uploaded documents agree with the application. Second, the inspection-prioritization studies select properties by predicted fire risk from historical data [4], [5]; they do not schedule inspections for applications already in a certification pipeline, where waiting time, deadlines, unresolved findings, and inspector availability matter. Third, the compliance-checking study [6] works on layouts, images, and regulatory clauses but does not manage an application through provisional and final certification. Fourth, the Document AI and symbol-spotting work [7], [8] is evaluated on general benchmarks rather than on the documents and drawings of an NOC process.

Within the sources reviewed here, no study connects these components in one workflow with human oversight and an auditable history. This is a statement about the reviewed sources and is not a claim that no such system exists.

B. Problem Statement

Applicants and inspectors in the Fire NOC process lack an integrated tool that (i) determines which requirements apply to a given building and explains why, (ii) flags missing, unreadable, or inconsistent documents before human review, (iii) offers preliminary analysis of fire-safety drawings, (iv) enforces a controlled path from application to final decision, (v) helps organize inspection workload transparently, and (vi) keeps a history in which later alteration can be detected. The problem addressed here is to design and evaluate such a system as a decision-support prototype in which AI findings are advisory, compliance rules are deterministic, and legally significant decisions stay with authorized personnel.

C. Research Questions

The work is guided by the following questions.

- RQ1: Can a deterministic, versioned rule engine generate a checklist that agrees with a checklist prepared manually from the official requirements for the same building?
- RQ2: How accurately can an OCR and NLP pipeline classify NOC supporting documents, extract key fields, and detect inconsistencies between a document and the application?
- RQ3: How precisely can a limited computer-vision model detect a defined set of fire-safety symbols in drawings, and how often must results be sent to manual review?
- RQ4: Does a transparent weighted-priority scheduler reduce waiting time and deadline violations compared with a first-come-first-served baseline?
- RQ5: Can a hash-chained audit trail detect modification, deletion, and reordering of historical events, and can the workflow layer reject illegal transitions?

D. Objectives

- Build a versioned, configurable checklist engine whose rules are extracted from verified official requirements and not invented.
- Build an OCR, classification, extraction, and cross-field validation pipeline with a human review step.
- Build a limited drawing-analysis prototype for a defined set of fire-safety symbols, with confidence values and manual-review flags.
- Implement a lifecycle state machine with role-based permissions and recorded reasons for returns and rejections.
- Implement a transparent weighted-priority scheduler and compare it with a baseline.
- Implement a hash-chained audit trail and test its ability to detect tampering.

E. Evaluation Plan

The research questions are answered by component-level tests and one end-to-end scenario test. Component tests use controlled data so that the correct answer is known. The end-to-end test follows a synthetic application from registration through checklist generation, document upload, review, inspection, rectification, and final decision, including at least one return for correction and one rejection, so that exception paths are exercised as well as the normal path. Measured results will be kept separate from illustrative values used in demonstrations. The metrics and data are listed in Section VI.

F. Distinction Between Existing and Proposed Methodology

Existing portals digitize the administrative workflow [2], [3]. Existing research addresses individual analytic tasks [4]–[8]. SmartFire-NOC combines both: it keeps the administrative workflow as the backbone and adds decision-support layers whose outputs feed human review. It differs from risk-prediction work in not claiming to predict fires, and from compliance-checking work in not claiming to certify compliance. Its contribution is the integration, the explicit separation between deterministic decisions and AI suggestions, and a measurable evaluation plan.

IV. PROPOSED METHODOLOGY

This section describes the method used for each part of the system. The modules are designed so that each can be tested alone, and the lifecycle and audit modules connect them into one process.

A. End-to-End Workflow

The overall process links the modules in sixteen steps. The applicant registers, signs in, and enters building and application information. The system generates the applicable checklist, and the applicant uploads the required documents. OCR and AI-assisted analysis process the documents, and the system flags missing information and inconsistencies, which the applicant then addresses. The inspector reviews the application and supporting material, and the application proceeds through the applicable provisional NOC process. Relevant drawings may undergo preliminary computer-vision analysis. The inspector receives an assignment or a scheduling recommendation, and findings and recommendations are recorded. Where required, the applicant submits rectification evidence, and the inspector reviews it and updates the findings. When the applicable requirements are met, the case proceeds to the authorized final decision. The audit trail records important actions at every stage.

The workflow supports validation, rework, rejection, resubmission, and other legitimate exceptions and does not assume that every case follows the same path.

B. Roles and Access

The system initially supports two roles. An applicant registers, creates and manages applications, enters building information, uploads documents, views results, tracks status, and uploads rectification evidence. An inspector reviews applications and documents, views checklist status and AI findings, manages inspection assignments, records findings and recommendations, and reviews rectification evidence. An administrator role for account management and rule configuration is planned for later. Public registration never grants inspector privileges; inspector accounts must be created or approved through a trusted administrative process.

C. Dynamic Checklist Engine

Requirements are stored as versioned rule records. Each rule has conditions over application attributes such as building type, new or existing status, industrial, commercial, or residential category, hazardous-material information, and building height, together with the requirement it produces and a reference to the official provision from which it was taken. The engine evaluates all rules deterministically against an application and returns a personalized checklist. Each item carries a state (required, submitted, pending, or reviewed) and an explanation of why it applies. The rule version used is stored with the evaluation so a past result can be reproduced. A language model is never used to create or change a requirement. Rules are extracted from the official Fire Department documentation and the governing Act [1], checked by a second reader against the source, and kept in a configurable format.

A rule has the general form: if the application satisfies a set of conditions on its attributes, then a stated requirement applies, with an explanation and a reference to its source. For example, a rule might read “if category is industrial and hazardous material is stored, then include requirement R and record the reason shown to the applicant.” This example only shows the structure; the actual conditions and requirements are taken from the verified official source and are not written from memory. When an official requirement changes, a new rule version is created and the earlier version is retained, so a checklist generated last year can still be explained. Rules that conflict or overlap are detected during rule authoring, and each rule is tested with at least one application that should trigger it and one that should not.

D. AI-Assisted Document Verification

The pipeline is: upload, file validation, OCR, classification, information extraction, cross-field validation, and review. File validation checks type, size, and file signature before processing. OCR extracts text from PDFs and images, using an open-source engine such as Tesseract [10] or an equivalent. Classification assigns each document to a configured category; a layout-aware model of the kind in [8] and a simple text-classification baseline will both be tried, and the simpler one will be kept unless the larger model measurably improves results. Extraction targets names, addresses, certificate numbers, and dates using patterns and named-entity recognition. Cross-field validation compares normalized extracted values with the application record using exact and approximate string matching, and flags missing information, mismatches, and unreadable files. Results show confidence values where the model supplies them. Applicants can correct or resubmit, and inspectors can review and resolve flags. A successful OCR result does not establish that a document is authentic or legally valid.

Failure handling is part of the design. A file that cannot be opened, is of an unsupported type, or is too large is rejected at upload with a clear message. A document whose OCR confidence is low is marked unreadable and the applicant is asked to resubmit a clearer copy. A document that is classified with low confidence, or as a category that does not match the checklist item it was uploaded for, is flagged and not silently re-labelled. An extracted field that disagrees with the application, for example a different owner name spelling or an expired certificate date, is shown to the inspector next to the application value, with the confidence and the location in the document, so that the inspector can accept, correct, or reject the flag. Every AI result stores the model version and the time of processing, so results can be reproduced and compared when models change.

E. Drawing / Floor Plan Analysis

This module is a limited prototype for preliminary analysis. Suitable PDFs are rasterized to images and preprocessed (grayscale conversion, denoising, resizing). An object-detection model locates a small, defined set of symbols, such as emergency-exit symbols, staircases, and fire-extinguisher symbols. Training and evaluation use a separately annotated sample set; public datasets such as [7] may help pre-training. Detections are returned with confidence values and bounding boxes that can be highlighted on the drawing, and detections below a threshold are flagged for manual review. Results are stored with the application. The feature is labelled as AI-assisted preliminary analysis. It does not certify compliance with fire codes and does not replace a qualified inspector.

The detection model is evaluated by matching predicted boxes to annotated boxes with an overlap threshold and counting true positives, false positives, and missed symbols for each class. Because a missed exit symbol and a false extra symbol have different consequences, both precision and recall are reported, along with the share of results sent to manual review at the chosen confidence threshold. A lower threshold sends fewer results to review but admits more wrong detections, so the threshold is chosen on validation data and is configurable. No number of detected symbols is converted into a pass or fail judgment.

F. NOC Lifecycle Control

The lifecycle is modelled as a state machine. The intended stages are application draft, submission, checklist and document review, payment verification where applicable, provisional NOC, inspection, findings and recommendations, rectification, verification, and final NOC decision. Only listed transitions are allowed, each is tied to a role, and returns, rejections, and requests for information require a recorded reason. Provisional and final NOC records are stored separately, and finalization is blocked while mandatory conditions remain incomplete. Table II shows illustrative transitions. The exact sequence and mandatory steps are configuration to be set from

the verified official procedure, and cases that need different steps must be supported.

A state machine makes illegal actions impossible by construction. The transition table lists, for every state, the next states that are allowed and the role that may trigger each one. The service layer checks this table before changing a record, and the check is made on the server regardless of what the interface shows. Each successful transition appends an entry to the status history with the actor, time, and reason, and also generates an audit event. Guards add conditions to specific transitions; for example, a move to the final decision requires that mandatory checklist items are reviewed, that no unresolved findings remain, and that the provisional record exists. Payment verification is included only where the official procedure requires it.

TABLE II. ILLUSTRATIVE LIFECYCLE TRANSITIONS

From	To	Actor
Draft	Submitted	Applicant
Submitted	Under review	Inspector
Under review	Returned for information (reason required)	Inspector
Returned	Resubmitted	Applicant
Under review	Provisional NOC decision	Authorized personnel
Provisional NOC	Inspection scheduled	Inspector
Inspection done	Rectification required	Inspector
Rectification required	Evidence submitted	Applicant
Evidence submitted	Final NOC decision	Authorized personnel

NOC approval and issuance stay under authorized human control. The system records and supports a decision and never makes it on the strength of AI output alone.

G. Inspection Scheduling

Each case receives a transparent priority score computed as a weighted sum of normalized factors, as in (1), where the weights are configurable and sum to one:

Each factor is normalized to the range zero to one so that the weights are comparable. For example, waiting time may be scaled by the longest current wait, and deadline proximity by the time remaining to the applicable deadline. The weights are stored as configuration, and the score and its factor values are shown to the inspector so the ordering can be understood and questioned.

$$S = w1W + w2D + w3U + w4C + w5B \qquad (1)$$

Here W is waiting time, D is deadline proximity, U is the amount of unresolved findings, C is case complexity, and B is a justified building-category factor. Statutory deadlines and inspector availability are handled as constraints, not as weights, so the score cannot silently override an official priority. Inspectors are assigned to time slots greedily in descending score order, subject to availability and, where reliable location data exist, travel time. Conflicts are shown, and assignment and rescheduling are supported and recorded. The score is an administrative ordering aid. It is not a fire-risk or building-safety rating. Performance will be compared against a first-come-first-served baseline.

A hypothetical illustration shows the mechanism. Suppose the weights are 0.30, 0.30, 0.20, 0.10, and 0.10. Case A has normalized factors 0.80, 0.90, 0.50, 0.40, and 0.60, giving S = 0.24 + 0.27 + 0.10 + 0.04 + 0.06 = 0.71. Case B has factors 0.40, 0.30, 0.80, 0.70, and 0.90, giving S = 0.12 + 0.09 + 0.16 + 0.07 + 0.09 = 0.53. Case A is scheduled first. These numbers are invented to show the arithmetic and are not measured results. The scheduler is compared with first-come-first-served ordering on simulated case sets using waiting time, overdue cases, and deadline violations.

H. Tamper-Evident Audit Trail

Significant actions in every module are recorded as audit events, including account and permission changes, application submission, document uploads, checklist evaluations, AI analysis events, inspection assignments, findings, and NOC transitions. Each event stores the actor ID, action type, entity type and ID, timestamp, change metadata, the previous event hash, and its own hash. The hash of event n is computed as $h(n) = \text{SHA-256}(h(n-1) \,\|\, \text{canonical}(\text{event } n))$, so changing, deleting, or reordering an earlier event breaks every later link. A verification routine recomputes the chain and reports the first inconsistency. Passwords, tokens, and unnecessary sensitive data are never logged. Hash chaining makes tampering detectable; it does not make records immutable [9]. An attacker who can write to storage and recompute the chain could rewrite it, so audit storage needs restricted write access, and periodically storing the latest hash in a separate location is a recommended safeguard.

Verification walks the chain from the first event. For each event it recomputes the digest from the stored fields and the previous digest and compares it with the stored value. A mismatch identifies the first altered or missing event, and the routine reports its position. Events must be serialized in a fixed canonical form before hashing, because a different field order would otherwise change the digest. Writes to the audit collection are made only by the audit service, which has no update or delete operation, and the audit interface is available only to authorized users. Failures that matter, such as repeated failed sign-ins or rejected unauthorized requests, are also recorded.

I. Authentication and Authorization

Passwords are hashed with bcrypt. Sessions use JSON Web Tokens [11], verified by middleware on every protected request. Access follows a role-based model [12], with extra ownership checks so an applicant can read only their own records. The backend validates all inputs and enforces every security-sensitive permission; protected front-end routes only improve usability. Secrets are kept in environment variables, files are served only after an authorization check, and the front end never connects to the database directly.

Registration creates only applicant accounts. Inspector accounts are created or approved by a trusted process, so no user can raise their own privilege. A token carries the user identifier and role and has a limited lifetime. When a token expires or is invalid, the interface returns the user to sign-in, and a valid token restores the session after a page refresh. Because stored passwords are hashed, they are never returned by the API, and file downloads check both the role and the ownership of the record before sending data.

V. SYSTEM ARCHITECTURE AND IMPLEMENTATION

A. Architectures in the Reviewed Studies

Firebird joins several datasets, trains risk models, and presents scores on an interactive map [4]. The compliance-checking study chains perception, semantic integration, and rule-based inference [6]. The DIGIT application organizes work as a role-based workflow with fee configuration and history [2]. These architectures serve different purposes and are not interchangeable, but they share two ideas used here: keeping model output separate from the rules that decide, and keeping a visible history of each case.

B. Architecture of SmartFire-NOC

The system has four layers. The presentation layer is a React and TypeScript single-page application styled with Tailwind CSS, with React Router and protected routes. The application layer is a Node.js, Express, and TypeScript REST API organized into routes, controllers, services, models, and middleware. The data layer is MongoDB Atlas accessed through Mongoose, with uploaded files kept in secure file storage and only references and metadata in the database. The AI layer consists of Python services for OCR, NLP, and computer vision, called by the backend through defined APIs. The rule engine, lifecycle controller, scheduler, and audit module are backend services. This arrangement keeps AI components replaceable and keeps security decisions in one place.

The backend is divided by responsibility. Route modules define the endpoints for authentication, applications, checklists, documents, verification, drawings, inspections, NOCs, and audit. Controllers handle requests and responses. Services hold the business logic, such as rule evaluation, transition checking, and scheduling. Models define the Mongoose schemas. Middleware performs authentication, role checks, input validation, and error handling. Keeping business logic in services means that the same rules apply whichever endpoint is used and that they can be unit tested without a web server.

The Python AI services run separately from the Node.js server and are reached through narrow interfaces that accept a file reference and return structured results with confidence values. This separation lets models be replaced or retrained without changing the application, and a failure in an AI service degrades the feature without stopping the rest of the system. The results are stored with the application and treated as advisory.

C. Data Design

The database holds the collections users, applications, checklistRules, documents, verificationResults, drawingAnalyses, inspections, inspectionFindings, nocs, and auditLogs. Further collections are added only when justified. The design uses unique email constraints, schema validation, references between related records, timestamps, explicit status values, indexes for common queries, and versioned rules.

Relationships are kept explicit. An application references its applicant; documents reference their application; verification results reference documents; drawing analyses, inspections, and NOC records reference the application; findings reference inspections; and audit logs reference the entity they describe and the actor. Indexes support the most common queries, such as the applications of one user, the pending items of one inspector, and the audit events of one record. Uploaded files are not stored in the database; the database holds the path, type, size, and a content digest used to detect later changes.

D. Implementation Plan

Development is incremental, and each phase is tested before the next begins:

- Phase 1: Finalize the responsive landing page.
- Phase 2: Implement authentication, MongoDB persistence, JWT, protected routes, and role-specific dashboards.
- Phase 3: Build application creation, editing, validation, and status display.
- Phase 4: Implement the checklist engine using verified requirements.
- Phase 5: Implement document upload, OCR, classification, extraction, and review.
- Phase 6: Implement the NOC lifecycle and controlled transitions.
- Phase 7: Build the drawing-analysis prototype.
- Phase 8: Implement inspection assignment, scheduling, findings, and rectification evidence.
- Phase 9: Integrate the audit trail and hash-chain verification throughout.

- Phase 10: Run integration and security testing, model evaluation, and interface refinement.

E. User Interface

The interface uses a light, professional government-technology style with white and off-white backgrounds, a deep red accent, charcoal text, and subtle orange highlights. It provides separate experiences for applicants and inspectors, shows only the actions and data relevant to the current role, and uses consistent loading, empty, success, and error states, accessible form controls, and responsive layouts.

F. Security Architecture

Security is applied at several points. Transport security protects data in transit. Authentication and role checks run before any controller. Ownership checks run inside the services so that an applicant can reach only their own records even if an identifier is guessed. Input is validated against schemas before it reaches the database. Uploaded files are checked for type and size and are stored outside the public web path. Secrets are held in environment variables. Logs avoid passwords, tokens, and unnecessary personal data. Frontend route protection improves the experience, but the backend enforces every permission that matters.

G. Deployment and Testing Approach

The prototype is intended to run on a development and a test environment, with the front end, back end, AI services, and database configured separately. Unit tests cover the rule engine, transition table, scheduler, and hash chain, since these are deterministic. Integration tests cover the sequence from sign-in to final decision. Security tests attempt cross-user access, role escalation, and expired-token use. The AI services are tested separately on held-out data, and their results are reported with the metrics in Section VI.

H. Distinction Between Existing and Proposed Architecture

The reviewed architectures each serve a single purpose. SmartFire-NOC treats deterministic rules, AI services, workflow control, scheduling, and auditing as separate modules connected to one application record and one audit chain. Its distinguishing property is that no AI output changes a case state by itself.

VI. RESULTS AND EVALUATION

A. Results Reported in the Reviewed Studies

Firebird reports true positive rates of up to 71% for fire prediction over more than 5,000 buildings [4]. The compliance-checking study reports layout-extraction pixel accuracy of 0.90 and safety-asset detection mAP of 0.91 [6]. The XGBoost study evaluates how well its models rank property fire risk on more than 72,000 properties [5]. LayoutLMv3 reports state-of-the-art results on several Document AI benchmarks [8].

B. Experimental Setup and Comparability

These figures come from different tasks, datasets, jurisdictions, and metrics. A true positive rate for fire prediction, a mean average precision for object detection, and a benchmark score for document understanding measure different things and should not be ranked against each other or used as a forecast for SmartFire-NOC.

C. Evaluation Proposed for This Project

This manuscript reports the design of SmartFire-NOC and its planned evaluation. No measured results are reported here. Testing will use controlled synthetic data and authorized data, since real application records are not available. Table III lists the planned measurements. Values shown in demonstration dashboards will be labelled as illustrative and never presented as measured results.

TABLE III. PLANNED EVALUATION

Component	Metrics	Data / baseline
Authentication and RBAC	Pass rate of role and ownership test cases	Role and permission test matrix
Checklist engine	Agreement with manually derived checklists	Scenarios built from verified requirements
OCR and classification	Character error rate; accuracy; macro-F1	Synthetic and authorized documents; simple text-classifier baseline
Extraction and mismatch detection	Field precision, recall, F1; detection and false-alarm rates	Documents with injected mismatches
Drawing analysis	Precision, recall, mAP per symbol class	Held-out annotated drawing set
Scheduling	Waiting time, overdue cases, deadline violations	Simulated case sets; first-come-first-served baseline
Workflow	Share of illegal transitions rejected	Transition test matrix
Audit trail	Detection rate for modified, deleted, and reordered events	Injected tampering
Data isolation	Unauthorized access attempts blocked	Cross-user access tests
Interface	Responsive layout and error handling checks	Manual test cases

For the learned components, results will be reported per class and per document type, with held-out test data kept apart from training data, and failure cases will be described instead of only averages. For the rule-based components, correctness will be checked against reviewed requirements, so a disagreement is treated as a defect in the rules or the code.

D. Test Data and Metric Definitions

Real application records are not available, so the test data are synthetic or authorized. Synthetic documents are generated from templates with controlled variation in layout, font, scan quality, and content, and a share of them carry injected faults such as a changed name, a swapped date, or a missing field. Because the faults are injected, the correct answer is known. Synthetic data cannot capture every variation in real documents, so the results will be described as measurements on a controlled sample.

Precision is the share of reported positives that are correct, recall is the share of true positives that are found, and the F1 score is their harmonic mean. For classification, macro-F1 averages the F1 score over classes so that rare classes count equally. Character error rate is the number of character edits needed to match the reference text divided by its length. For detection, a prediction counts as correct when its overlap with an annotated box exceeds a stated threshold, and mean average precision summarizes precision across recall levels and classes. For scheduling, the measures are the mean waiting time, the number of overdue cases, and the number of deadline violations over simulated case sets.

E. Threats to Validity

- Synthetic data may be easier or differently distributed than real submissions, so measured accuracy may not carry over.
- A small annotated drawing set limits the reliability of detection estimates; results will be reported with the sample size.
- Simulated scheduling cases reflect assumptions about arrival patterns and deadlines that may differ from a real department.
- Rule correctness is judged against the same reviewed requirements that the rules were built from, so an error in reading the source would not be caught; an independent second review reduces this risk.
- Tampering tests use the attacker models chosen by the authors, which do not cover every possible attack.

F. Interpretation and Project Relevance

The reviewed studies show that each analytic component is feasible in its own setting [4]–[8]. They do not show that the combination will work on NOC documents, drawings, and workflows. The planned evaluation is designed to answer that question component by component, so that weak parts, such as drawing analysis on low-quality scans, can be reported honestly and kept advisory.

VII. LIMITATIONS AND FUTURE SCOPE

A. Limitations of the Reviewed Studies

The risk-prediction studies depend on historical incident and property data from a particular city or department [4], [5], so their models may not transfer. The compliance-checking study targets operational buildings and uses specialized perception and reasoning components [6]. The Document AI and symbol-spotting results come from general benchmarks [7], [8]. The portal descriptions are public summaries and do not document internal analytics [2], [3].

B. Additional Limitations Relevant to SmartFire-NOC

- It is an academic prototype with no official affiliation, and it must not be presented as a government service.
- Checklist correctness depends on the accuracy of the rules extracted from official sources and on keeping them current.
- OCR quality falls on poor scans, and neither OCR nor classification can establish that a document is genuine.
- Drawing analysis covers only a few symbols, cannot certify code compliance, and may fail on unusual drawing styles.
- The scheduling score is an administrative aid and not a safety rating, and weights chosen without real data may need tuning.
- Hash chaining is tamper-evident and not an absolute guarantee of immutability.
- Synthetic test data limits how far results generalize to real records.
- Uploaded documents are personal data and need strict access control and careful retention rules.

C. Future Scope

- Administrator tools for rule management, inspector approval, and rule version review.
- Digital signatures and verifiable certificates, and external anchoring of audit-chain hashes.
- Integration with official portals and payment systems through approved interfaces.
- Support for Marathi and Hindi documents in OCR and extraction.
- A larger annotated drawing set, more symbol classes, and evaluation with practising inspectors.
- Risk-informed prioritization, only after suitable data are available and the approach is approved by the authority.
- A mobile inspection tool with offline capture, and a usability study with real users.

These directions are proposals for this project and are not claims made by the reviewed papers.

D. Ethical and Legal Considerations

The system handles personal and property documents, so access control and data minimization are part of its design. It must not be presented as a government service, and its outputs must not be taken as a legal determination. AI suggestions can be wrong and can appear confident; the interface therefore labels them as suggestions and keeps the human reviewer responsible for the decision. Applicants should be able to see why a requirement was applied or a flag was raised and to correct or resubmit documents, so that automation does not remove the opportunity to respond. Any use with real records would require the approval of the competent authority and compliance with applicable data-protection rules.

VIII. CONCLUSION

This paper proposed SmartFire-NOC, an intelligent platform for Fire Department inspection and NOC management that links a rule-based checklist engine, AI-assisted document verification, preliminary drawing analysis, controlled provisional-to-final NOC tracking, transparent inspection scheduling, and a hash-chained audit trail. Its design keeps compliance decisions deterministic, treats AI output as a suggestion for review, and leaves approval with authorized personnel.

The literature shows that each analytic component is feasible on its own, and that online NOC portals exist, but within the sources reviewed here these pieces are not combined in one auditable workflow. SmartFire-NOC is an attempt to do that in a prototype, with a defined evaluation plan covering authorization, rule correctness, document and drawing analysis, scheduling, workflow validity, and audit integrity. Measured results will determine which components are reliable enough to be used and which should remain advisory. The system supports preliminary decision-making and record-keeping; it does not replace qualified fire officers or the official approval process.

REFERENCES

[1] Government of Maharashtra, “The Maharashtra Fire Prevention and Life Safety Measures Act, 2006 (Mah. Act No. III of 2007),” as amended, including the Maharashtra Fire Prevention and Life Safety Measures (Amendment) Act, 2023 (Mah. Act No. 24 of 2023).

[2] eGovernments Foundation, “DIGIT Fire NOC.” [Online]. Available: https://digit.org/digit-urban-fire-noc (accessed Oct. 22, 2026).

[3] Centre for Good Governance, Hyderabad, “Comprehensive online application for Fire Department,” e-Governance Projects. [Online]. Available: https://cgg.gov.in/it_project/1618/ (accessed Oct. 22, 2026).

[4] M. Madaio, S.-T. Chen, O. L. Haimson, W. Zhang, X. Cheng, M. Hinds-Aldrich, D. H. Chau, and B. Dilkina, “Firebird: Predicting fire risk and prioritizing fire inspections in Atlanta,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, USA, Aug. 2016, doi: 10.1145/2939672.2939682.

[5] R. Dey and A. Fern, “Data-driven structural fire risk prediction for city properties,” in Proc. AAAI Conf. Artificial Intelligence, vol. 38, no. 21, 2024, doi: 10.1609/aaai.v38i21.30325.

[6] D. Chen, L. Chen, Y. Yang, Q. Lu, C. Hancock, R. Lock, and S. Sølvsten, “A knowledge-driven approach for automated fire safety compliance checking in operational buildings,” Knowledge-Based Systems, vol. 336, Art. no. 115156, Mar. 2026, doi: 10.1016/j.knosys.2025.115156.

[7] Z. Fan, L. Zhu, H. Li, X. Chen, S. Zhu, and P. Tan, “FloorPlanCAD: A large-scale CAD drawing dataset for panoptic symbol spotting,” in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2021, pp. 10128–10137.

[8] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, “LayoutLMv3: Pre-training for document AI with unified text and image masking,” in Proc. 30th ACM Int. Conf. Multimedia, 2022, doi: 10.1145/3503161.3548112.

[9] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” ACM Trans. Information and System Security, vol. 1, no. 3, pp. 159–176, 1999.

[10] R. Smith, “An overview of the Tesseract OCR engine,” in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.

[11] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.

[12] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.